Final Project: Image Object Detection

測量115 F64111134 施淳譯

A. YOLOv13 Model Introduction

YOLO (You Only Look Once) is a single-stage object detection model that performs object localization and classification in one forward pass. It is well known for its lightweight architecture, fast inference speed, and strong compatibility with various software frameworks and embedded systems. In contrast, models such as R-CNN employ a two-stage detection pipeline, while DETR introduces transformer-based architectures for object detection. The primary advantage of YOLO lies in its compact model size and real-time inference capability, making it suitable for edge devices and real-world applications. However, YOLO also has certain limitations. Its performance tends to degrade in scenarios involving dense object distributions, and it generally achieves lower detection accuracy compared to two-stage detection models.

YOLOv13 was published on June 21, 2025, and represents the latest iteration of the YOLO family, proposed by iMoon-Lab at Tsinghua University, Beijing. The architecture of YOLOv13 is illustrated below.

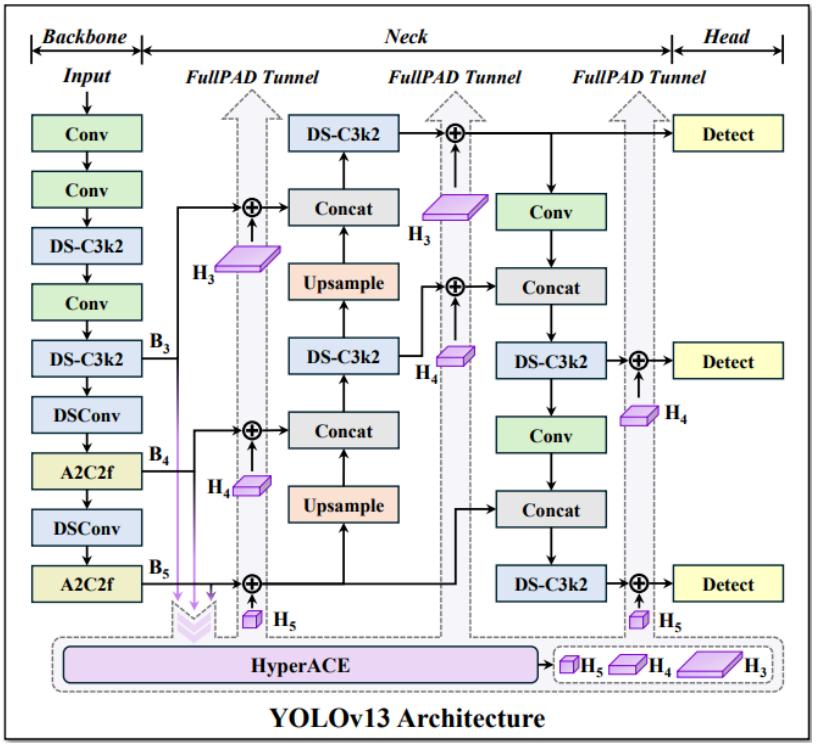


Figure. 1. YOLOv13 Architecture

YOLOv13 adopts the same overall architecture as previous YOLO versions, consisting of three main components: Backbone, Neck, and Head. One of the key characteristics of YOLOv13 is the use of Depthwise Separable Convolution (DSConv), which significantly reduces the number of parameters while maintaining performance comparable to standard convolution layers. In addition, YOLOv13 introduces the HyperACE block, which is designed to model and enhance the relationships among objects within an image, enabling more effective feature representation. The detailed structure of this block is illustrated below.

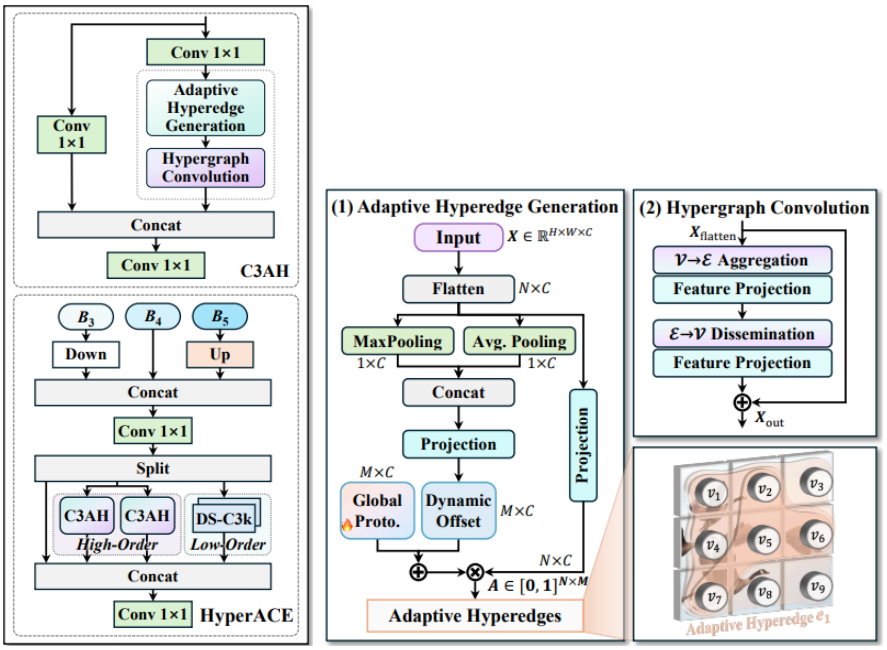


Figure. 2. Hyperedge Block Architecture

In HyperACE, B3, B4, and B5 are used as inputs, representing high-, medium-, and low-resolution feature maps, which correspond to low-, medium-, and high-level semantic information, respectively. These multi-scale features are first aligned and concatenated to integrate information across all resolution levels. After feature fusion, the combined feature map is split into two main branches: the High-Order branch and the Low-Order branch. In the High-Order branch, K parallel C3AH blocks are employed to extract multiple potential correlations, where each correlation can be interpreted as a hyperedge. Within each C3AH block, the input feature is divided into two sub-branches: one branch preserves the original feature through a lateral connection, while the other branch feeds features into the hypergraph computation module. The Adaptive Hyperedge Generation module determines which patches (vertices) belong to the same hyperedge, dynamically constructing high-order relationships based on the input feature. Subsequently, the Hypergraph Convolution module propagates high-level semantic information across the constructed hypergraph, updating the vertex features by aggregating group-level contextual information. In contrast, the Low-Order branch utilizes DS-C3k blocks, which rely on convolutional operations to capture local and low-level features efficiently. Finally, the outputs of the High-Order branch and Low-Order branch are concatenated together, and a shortcut connection is added to preserve the original feature information, ensuring feature stability and preventing information loss during high-order modeling. In summary, HyperACE enables the model to capture high-order semantic relationships that are difficult to model using conventional convolution alone, which is particularly beneficial in complex traffic scenes.

YOLOv13 achieves better mAP performance compared to previous YOLO models, and in some model variants, it requires fewer FLOPs than earlier versions.

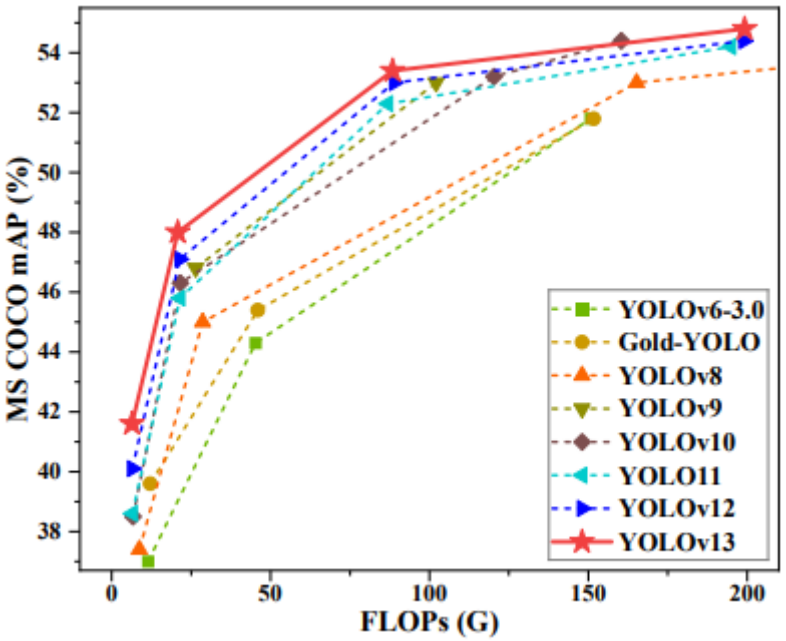


Figure 3. Comparison of YOLO model variants. From left to right, the models correspond to the n, s, l, and x series, respectively, representing configurations with increasing numbers of parameters.

B. Training Dataset and parameters

The dataset used in this study is a road traffic signal dataset, composed of two parts: a dataset provided by the laboratory and a dataset collected independently. The laboratory dataset contains a total of 17,390 images, with approximately 400 images per class. All images in this dataset were collected on a university campus during daytime, resulting in relatively balanced class distributions. The self-collected dataset consists of 3,397 images captured in real-world road environments. Labeling with Labelme software. In this dataset, the number of images per class is imbalanced, as certain traffic signals are rarely encountered and difficult to collect. Additionally, some signal classes appear only in the self-collected dataset and are absent from the laboratory dataset. Each dataset was individually split into training and testing sets, with 80% of the data used for training and 20% for testing on a per-class basis. Afterward, the two datasets were combined to form the final dataset used for model training and evaluation. The training sets and testing sets from both datasets were merged separately. No explicit class re-weighting was applied, and class imbalance remains a limitation of this study.

Some of the training hyperparameters and data augmentation parameters I use are shown below.

Parameters	Fine-tuning (Pre-trained model)	Transfer Learning (Stage1/Stage2)	Describe
epochs	300	100/100	Training epochs
batch	24	24	Batch size

imgsz	640	640	Training image size
max_det	50	50	Maximum number of detections allowed per image
freeze	0	9/0	Freeze first N layers
erasing	0.1	0.1	Random erasing some part of image probability
fliplr	0.0	0.0	Random horizontal flip rate
hsv_h	0.015	0.015	Adjust the hue of the image between -+0.015
hsv_s	0.7	0.7	Alters the saturation of the image between -+0.7
hsv_v	0.4	0.4	Modifies the brightness of the image between -+0.4
scale	0.1	0.1	Random scale the image between -+0.1
translate	0.1	0.1	Random translate the image between -+0.1
degrees	5	5	Random rotation within -+5 degrees
mosaic	0.3	0.3	The ratio of combines four training images into one.
close_mosaic	20	20	Disables mosaic data augmentation in the last N epochs.
optimizer	auto	AdamW	Optimizer for training.
lr0	0.01	0.01/0.001	Initial learning rate

Use the below code for training.

```
from ultralytics import YOLO

model = YOLO('yolov13{n/s/l/x}.pt')

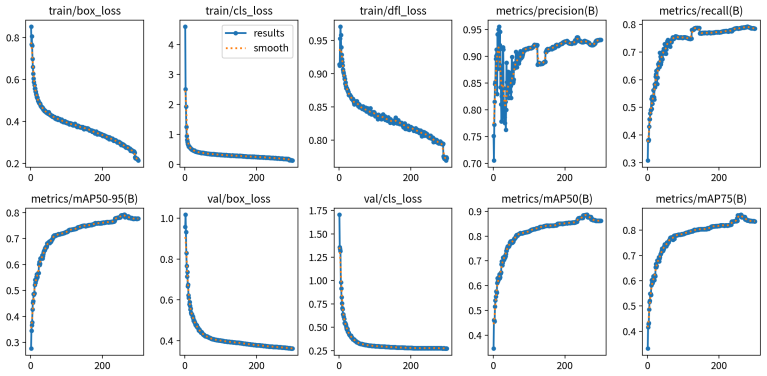
results = model.train({hyperparameters})
```



Figure. 4. Training image example

C. Yolo Model Training -- Fine Tune Pre-Train Model

In this work, fine-tuning refers to updating all model parameters simultaneously, whereas transfer learning is implemented as a two-stage training strategy with partial layer freezing. In this part, a pre-trained YOLOv13n model trained on the COCO50 dataset was used as the initialization. The model was fine-tuned using a small learning rate, while the optimizer was automatically selected by the YOLO framework. The training was conducted for 300 epochs. Relatively conservative data augmentation settings were applied, as some traffic signals are very small and difficult for the model to learn, and excessive augmentation may degrade detection performance. The corresponding training curves and experimental results are presented below.



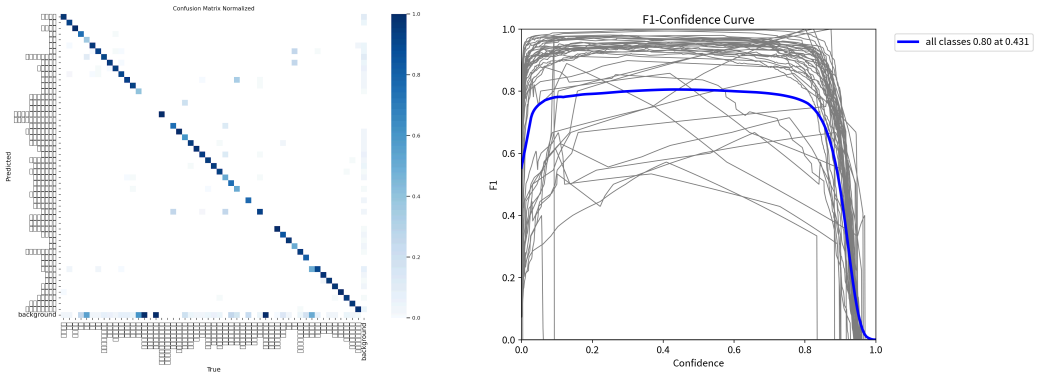


Figure. 5. Training curve, Confusion matrix and F1-Confidence Curve on Fine-tune yolov13 model

D. Yolo Model Training -- Transfer Learning

Transfer learning techniques were applied to leverage the knowledge of a pre-trained model, particularly in the convolutional layers. In Stage 1, the first nine layers, which are located before the HyperACE module and consist mainly of convolutional and attention-based blocks, were frozen. The model was trained for 100 epochs to adapt the higher-level layers while preserving low-level feature representations. In Stage 2, the learning rate was reduced, and the AdamW optimizer was adopted. All layers were unfrozen to allow full parameter updates, and the model was further trained for 100 epochs. The experimental results are presented below.

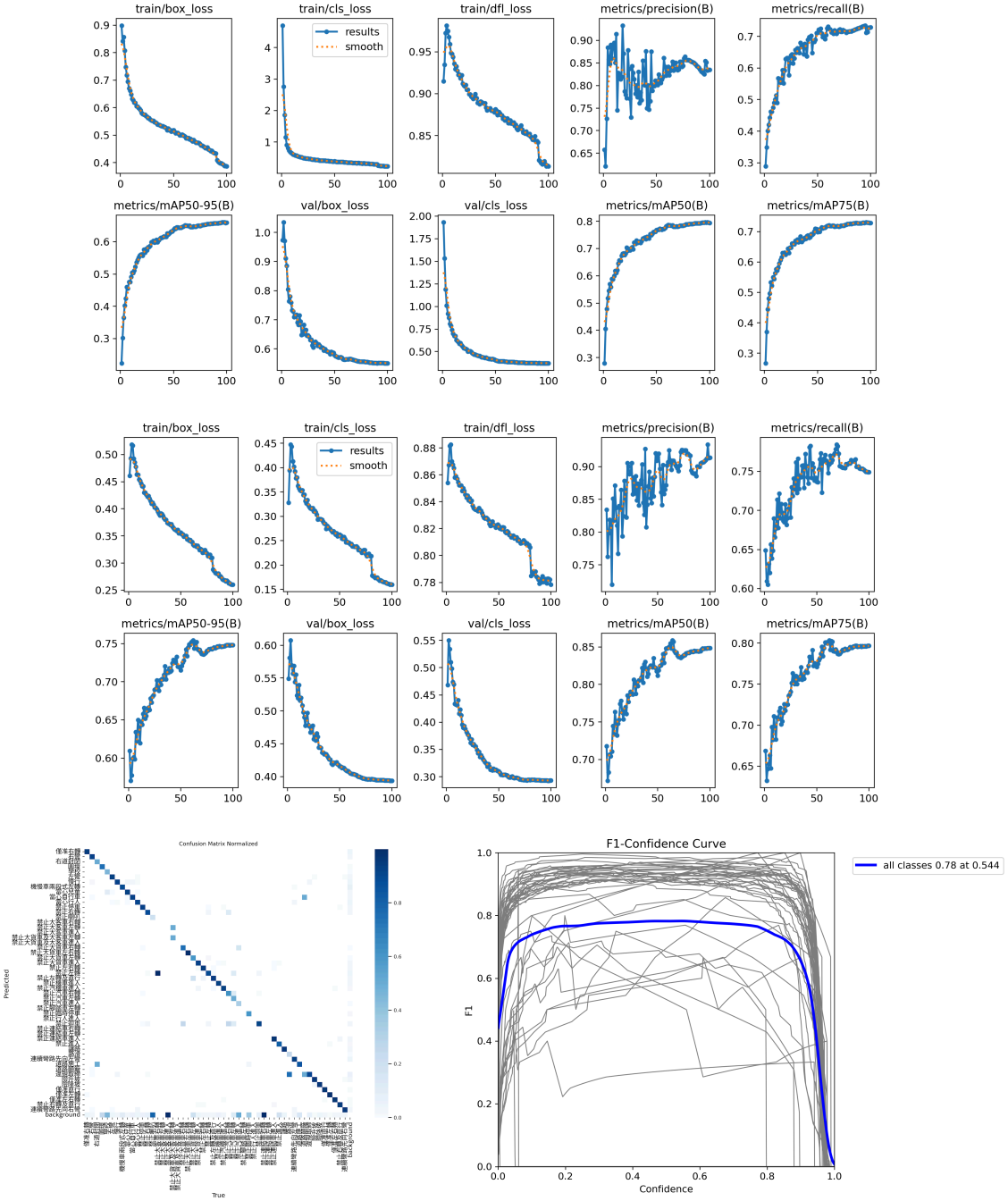


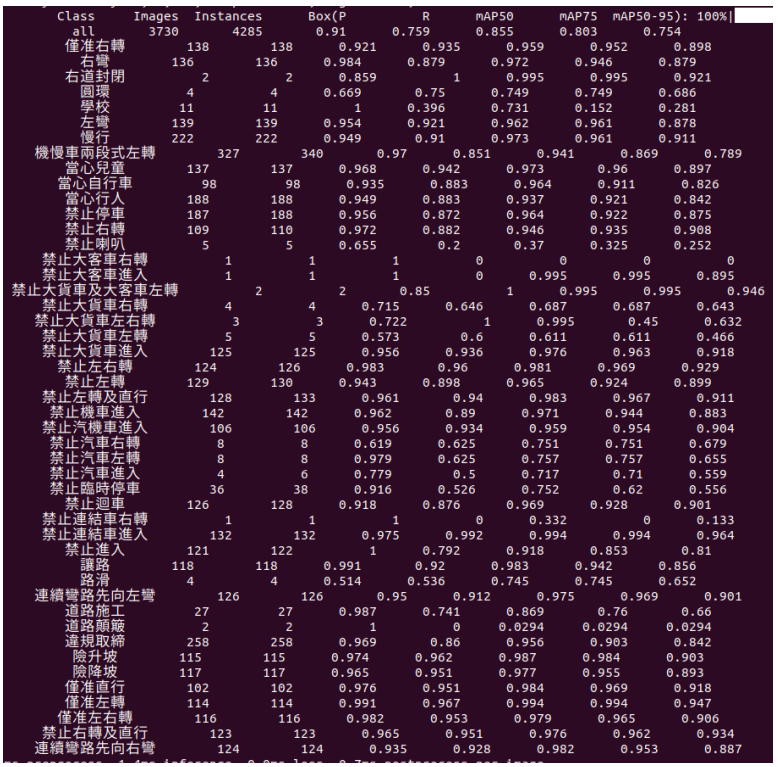
Figure. 6. Training curves for Stage 1 and Stage 2 in the transfer learning process, along with the confusion matrix and F1-score obtained in Stage 2.

E. Validation and Conclusion

Since YOLOv13 does not support Chinese characters, it was necessary to modify parts of the output and visualization code. Consequently, the confusion matrix in the fine-tuning stage does not display Chinese class names. Nevertheless, the class order and label mapping remain consistent with those shown in the confusion matrix of the transfer learning stage, where the code modifications were applied. From the training curves presented above, it can be observed that both the fine-tuning and transfer learning approaches exhibit similar loss convergence behavior. However, in terms of mAP, the fine-tuned model achieves better performance across multiple evaluation benchmarks. Two validation results and one validation batch are shown below. The results indicate that both models perform well on traffic signal classes with a large number of training samples, while their performance deteriorates for classes with limited training data. Notably, the transfer learning model achieves higher recall and mAP on the validation set. Overall, these results suggest that both models achieve comparable overall performance. However, the transfer learning approach significantly reduces training time compared to full fine-tuning, making it a more efficient solution when computational resources or training time are limited. And also have better validation results. I think the transfer learning model gets a better result in this work.



Class	Images	Instances	Box(P	R	mAP50	mAP75	mAP50-95): 100%
all	3662	4191	0.914	0.749	0.815	0.798	0.744
右彎	142	142	0.982	0.908	0.967	0.939	0.889
右道封閉	4	4	0.856	1	0.995	0.995	0.865
圓環	6	6	1	0.597	0.691	0.691	0.632
學校	6	6	1	0	0.0321	0	0.014
左彎	133	133	0.958	0.925	0.968	0.964	0.905
慢行	227	227	0.968	0.912	0.976	0.974	0.927
機慢車兩段式左轉	305	312	0.949	0.907	0.966	0.912	0.82
當心兒童	142	143	0.963	0.906	0.965	0.95	0.9
當心自行車	96	96	0.92	0.969	0.971	0.955	0.869
當心行人	180	180	0.94	0.875	0.937	0.905	0.856
禁止停車	182	182	0.976	0.89	0.973	0.949	0.905
禁止右轉	108	108	0.985	0.935	0.984	0.984	0.956
禁止喇叭	7	7	0.851	0.429	0.731	0.582	0.463
禁止大客車右轉	2	2	1	0.827	0.995	0.995	0.897
禁止大客車進入	1	1	1	0	0	0	0
禁止大貨車及大客車左轉	1	1	0.697	1	0.995	0.995	0.995
禁止大貨車及大客車進入	1	1	0.626	1	0.995	0.995	0.895
禁止大貨車右轉	4	4	0.366	0.25	0.465	0.465	0.393
禁止大貨車左轉	4	4	0.644	0.75	0.691	0.691	0.635
禁止大貨車進入	125	125	0.961	0.936	0.973	0.968	0.92
禁止左右轉	125	128	0.977	0.938	0.971	0.969	0.926
禁止左轉	127	129	0.983	0.895	0.971	0.95	0.932
禁止左轉及直行	117	117	0.991	0.971	0.994	0.994	0.957
禁止機車進入	141	141	0.97	0.92	0.967	0.95	0.893
禁止汽機車進入	106	106	0.956	0.934	0.961	0.956	0.92
禁止汽車右轉	7	7	0.849	0.857	0.86	0.86	0.789
禁止汽車左轉	8	8	1	0.483	0.73	0.73	0.674
禁止汽車進入	12	14	1	0.133	0.444	0.365	0.298
禁止臨時停車	38	43	0.968	0.712	0.846	0.79	0.675
禁止行人進入	1	1	0	0	0	0	0
禁止迴車	112	114	0.964	0.927	0.974	0.954	0.915
禁止連結車右轉	1	1	1	0	0	0	0
禁止連結車進入	132	132	0.992	0.982	0.994	0.994	0.974
禁止進入	102	103	0.977	0.842	0.937	0.912	0.863
讓路	118	118	0.978	0.958	0.993	0.972	0.895
路滑	1	1	1	0	0.332	0.332	0.199
連續彎路先向左彎	126	126	0.992	0.932	0.979	0.972	0.915
道路施工	21	21	1	0.497	0.686	0.575	0.521
道路顛簸	3	3	0.936	0.333	0.35	0.35	0.315
違規取締	250	250	0.949	0.887	0.968	0.946	0.89
陡升坡	116	116	0.991	0.94	0.98	0.98	0.911
陡降坡	117	117	0.953	0.949	0.967	0.96	0.9
僅準直行	102	102	0.976	0.98	0.987	0.987	0.94
僅準右轉	132	132	0.941	0.965	0.987	0.982	0.937
僅準左轉	114	114	0.976	0.982	0.994	0.994	0.957
僅準左右轉	116	116	0.982	0.955	0.984	0.958	0.923
禁止右轉及直行	123	123	1	0.949	0.981	0.981	0.953
連續彎路先向右彎	124	124	0.953	0.992	0.989	0.974	0.922



Class	Images	Instances	Box(P	R	mAP50	mAP75	mAP50-95): 100%
all	3730	4285	0.91	0.759	0.855	0.803	0.754
僅准右轉	138	138	0.921	0.935	0.959	0.952	0.898
右彎	136	136	0.984	0.879	0.972	0.946	0.879
右道封閉	2	2	0.859	1	0.995	0.995	0.921
圓環	4	4	0.669	0.75	0.749	0.749	0.686
學校	11	11	1	0.396	0.731	0.152	0.281
左彎	139	139	0.954	0.921	0.962	0.961	0.878
慢行	222	222	0.949	0.91	0.973	0.961	0.911
機慢車兩段式左轉	327	340	0.97	0.851	0.941	0.869	0.789
當心兒童	137	137	0.968	0.942	0.973	0.96	0.897
當心自行車	98	98	0.935	0.883	0.964	0.911	0.826
當心行人	188	188	0.949	0.883	0.937	0.921	0.842
禁止停車	187	188	0.956	0.872	0.964	0.922	0.875
禁止右轉	109	110	0.972	0.882	0.946	0.935	0.908
禁止喇叭	5	5	0.655	0.2	0.37	0.325	0.252
禁止大客車右轉	1	1	1	0	0	0	0
禁止大客車進入	1	1	1	0	0.995	0.995	0.895
禁止大貨車及大客車左轉	2	2	0.85	1	0.995	0.995	0.946
禁止大貨車右轉	4	4	0.715	0.646	0.687	0.687	0.643
禁止大貨車左右轉	3	3	0.722	1	0.995	0.45	0.632
禁止大貨車左轉	5	5	0.573	0.6	0.611	0.611	0.466
禁止大貨車進入	125	125	0.956	0.936	0.976	0.963	0.918
禁止左右轉	124	126	0.983	0.96	0.981	0.969	0.929
禁止左轉	129	130	0.943	0.898	0.965	0.924	0.899
禁止左轉及直行	128	133	0.961	0.94	0.983	0.967	0.911
禁止機車進入	142	142	0.962	0.89	0.971	0.944	0.883
禁止汽機車進入	106	106	0.956	0.934	0.959	0.954	0.904
禁止汽車右轉	8	8	0.619	0.625	0.751	0.751	0.679
禁止汽車左轉	8	8	0.979	0.625	0.757	0.757	0.655
禁止汽車進入	4	6	0.779	0.5	0.717	0.71	0.559
禁止臨時停車	36	38	0.916	0.526	0.752	0.62	0.556
禁止迴車	126	128	0.918	0.876	0.969	0.928	0.901
禁止連結車右轉	1	1	1	0	0.332	0	0.133
禁止連結車進入	132	132	0.975	0.992	0.994	0.994	0.964
禁止進入	121	122	1	0.792	0.918	0.853	0.81
讓路	118	118	0.991	0.92	0.983	0.942	0.856
路滑	4	4	0.514	0.536	0.745	0.745	0.652
連續彎路先向左彎	126	126	0.95	0.912	0.975	0.969	0.901
道路施工	27	27	0.987	0.741	0.869	0.76	0.66
道路顛簸	2	2	1	0	0.0294	0.0294	0.0294
違規取締	258	258	0.969	0.86	0.956	0.903	0.842
陡升坡	115	115	0.974	0.962	0.987	0.984	0.903
陡降坡	117	117	0.965	0.951	0.977	0.955	0.893
僅准直行	102	102	0.976	0.951	0.984	0.969	0.918
僅准左轉	114	114	0.991	0.967	0.994	0.994	0.947
僅准左右轉	116	116	0.982	0.953	0.979	0.965	0.906
禁止右轉及直行	123	123	0.965	0.951	0.976	0.962	0.934
連續彎路先向右彎	124	124	0.935	0.928	0.982	0.953	0.887

Figure. 7. The first image is the fine-tuned method result and the second is the transfer method result





Figure. 8. The first image is validation label, the second one is fine-tuned model predict and the last one is pre-trained model result

F. Reference

1. YOLO parameters: https://docs.ultralytics.com/usage/cfg/#train-settings
2. YOLOv13 github: https://github.com/iMoonLab/yolov13
3. YOLOv13 paper: https://arxiv.org/abs/2506.17733